

Revisión Técnica de Artículo:

Simulación de Multitudes en Tiempo Real mediante Boids y Spatial Hashing en GPU

Proyecto Final
Gráficas Computacionales

Integrantes: Martha Martinez Loa y Andres Velasco

Resumen

En este trabajo analizamos una técnica para simular enjambres y multitudes en tiempo real llevando el algoritmo clásico de Boids (Reynolds, 1986) a la GPU usando un *Spatial Hash Grid*. Evaluamos cómo este enfoque resuelve el problema de rendimiento al pasar de una búsqueda exhaustiva $\mathcal{O}(N^2)$ a una consulta local $\mathcal{O}(N)$ revisamos sus límites cuando la densidad de agentes cambia y comentamos su aplicación práctica en nuestro propio motor de enjambres.

1. Introducción y Antecedentes

El modelo de Boids de Craig Reynolds es el punto de partida estándar para simular comportamiento de bandadas o peces en computación gráfica. La idea es sencilla: cada agente toma decisiones en función de lo que hacen sus vecinos cercanos usando tres reglas básicas:

1. **Separación:** Mantener una distancia mínima para no chocar.
2. **Alineación:** Ajustar la dirección para ir hacia donde se mueve el grupo.
3. **Cohesión:** Caminar hacia el punto medio del grupo de vecinos.

El problema de este modelo en su forma más simple es que cada agente necesita revisar la posición de todos los demás para saber quiénes son sus vecinos. Esto genera una complejidad de $\mathcal{O}(N^2)$, lo que significa que al pasar de unos 50 o 100 agentes a escenarios con 200 o más, el tiempo de cálculo en CPU dispara los tiempos de render y los FPS caen por debajo de los 30 cuadros por segundo requeridos para interacción en vivo.

2. Resumen de la Solución Técnica

El artículo resuelve este cuello de botella reestructurando la forma en que los agentes buscan a sus vecinos mediante una grilla espacial uniforme (*Spatial Hash Grid*) que se procesa en paralelo.

2.1. Indexación por Spatial Hashing

En lugar de revisar la escena completa, el espacio 3D se divide en celdas de tamaño S , donde S es al menos igual al radio de percepción r de los agentes. La posición tridimensional $P(x, y, z)$ de cada boid se traduce a un índice de celda en una tabla de hash usando la función:

$$Key(P) = \left\lfloor \frac{x}{S} \right\rfloor + \left\lfloor \frac{y}{S} \right\rfloor \cdot M_x + \left\lfloor \frac{z}{S} \right\rfloor \cdot M_y \quad (1)$$

Gracias a esto, en cada fotograma un agente solo necesita consultar su propia celda y las 26 celdas contiguas (un bloque de $3 \times 3 \times 3$). Esto acorta la búsqueda de vecinos de $N - 1$ elementos a un subconjunto pequeño, logrando un comportamiento cercano a $\mathcal{O}(N)$.

2.2. Paralelismo y Renderizado

El artículo aprovecha que el cálculo de fuerzas de cada agente es independiente para ejecutar el paso de integración en la GPU. Al combinar esta grilla con *Instanced Rendering*, el costo de invocar el dibujado en pantalla (*Draw Calls*) se reduce a una sola llamada sin importar cuántos agentes haya en la simulación.

3. Análisis Crítico y Discusión

3.1. Puntos Fuertes

- **Escalabilidad práctica:** Permite sostener 60 FPS estables en tiempo real con cientos o miles de agentes en pantalla.
- **Bajo consumo de memoria:** La estructura de datos de la grilla es ligera y fácil de reiniciar entre fotogramas.
- **Adaptabilidad:** El algoritmo no depende de un pipeline visual específico; se puede integrar tanto con shaders GLSL estándar como con arquitecturas basadas en WebGPU.

3.2. Limitaciones y Casos Límite

- **Agrupamiento extremo:** Si por una fuerza de cohesión muy alta todos los agentes se concentran dentro de una o dos celdas, el beneficio del hash disminuye y la complejidad vuelve a acercarse localmente a $\mathcal{O}(N^2)$.
- **Reconstrucción frecuente:** Dado que los agentes están en constante movimiento, la grilla debe limpiarse y rellenarse por completo en cada cuadro, lo que añade un pequeño costo fijo de CPU si la reconstrucción no se hace directamente en cómputo GPU.

4. Aplicabilidad a Nuestro Proyecto

Implementamos la estructura de Spatial Hash Grid analizada en el artículo dentro de nuestro proyecto final usando **Three.js**. Esta arquitectura nos permitió mantener 60 FPS estables con más de 200 agentes simulados en paralelo y animados mediante *Vertex Animation Textures* (VAT) ejecutados en el Vertex Shader.

5. Conclusión

El artículo muestra cómo una estructura de datos clásica como el hash espacial, combinada con un pipeline de renderizado por instancias, resuelve de forma limpia el problema de escalabilidad del modelo Boids. Es una solución directa, fácil de implementar y muy efectiva para entornos interactivos en tiempo real.